

Lesson 1: C Programming Bit Manipulation & (FSM)

Thinh Tang - Trieu Nguyen

Bộ môn Điện tử - Viễn thông, Khoa Điện - Điện tử
Trường Đại học Bách khoa - ĐHQG-HCM

Email liên hệ: mliot@hcmut.edu.vn

Machine Learning & IoT Lab (ML IoT)
Ho Chi Minh City University of Technology (HCMUT)

1. Kiến trúc Máy tính & Khái niệm Bit
2. Kiểu dữ liệu chuẩn hóa & Thao tác Bit
3. Quản lý biến & Con trỏ trong nhúng
4. Compiler Optimization & Từ khóa volatile
5. Máy trạng thái hữu hạn (FSM) & Tư duy Non-blocking
6. Bài tập thực hành & Demo

Contact Information

Address: 403.1 BK.B6, HCMUT Campus 2

Email:

nkloi@hcmut.edu.vn

mlandiotlab@gmail.com

Website: mliotlab.com

Facebook: facebook.com/hcmut.ml.iot.lab

YouTube: youtube.com/@mliotlab

Kiến trúc Von Neumann

- ▶ Bộ nhớ lưu trữ dữ liệu (Data) và lệnh (Instruction) chung nhau.
- ▶ Sử dụng chung một hệ thống Bus.
- ▶ **Ưu điểm:** Phần cứng đơn giản, linh hoạt phân chia vùng nhớ.
- ▶ **Nhược điểm:** Nghẽn cổ chai Bus (Von Neumann Bottleneck) - CPU không thể vừa đọc lệnh vừa đọc dữ liệu cùng lúc.

Kiến trúc Harvard

- ▶ Bộ nhớ lưu trữ dữ liệu và lệnh tách biệt hoàn toàn.
- ▶ Sử dụng các hệ thống Bus riêng biệt.
- ▶ **Ưu điểm:** Cho phép CPU truy cập lệnh và dữ liệu song song → Tốc độ thực thi nhanh hơn.
- ▶ **Nhược điểm:** Thiết kế mạch phần cứng phức tạp và đắt tiền hơn.

Kiến trúc CPU hiện đại thường dùng: Hầu hết vi điều khiển hiện đại (như **ARM Cortex-M**, x86) sử dụng kiến trúc **Harvard cải tiến (Modified Harvard Architecture)**: Tách biệt bộ nhớ đệm Cache L1 (L1 Instruction Cache & L1 Data Cache) để tối ưu tốc độ, nhưng dùng chung bộ nhớ chính (Flash/RAM) để tiết kiệm chân bus vật lý và tài nguyên.

MSB (Most Significant Bit)

- ▶ Bit có **trọng số lớn nhất**.
- ▶ Nằm ở vị trí ngoài cùng bên trái trong một chuỗi bit.
- ▶ Trong số có dấu (Signed Integer), MSB chính là bit dấu (Sign bit: 0 = Dương, 1 = Âm).

LSB (Least Significant Bit)

- ▶ Bit có **trọng số nhỏ nhất**.
- ▶ Nằm ở vị trí ngoài cùng bên phải trong một chuỗi bit.
- ▶ Dùng để kiểm tra tính chẵn lẻ của một số ($LSB = 0 \rightarrow$ Chẵn, $LSB = 1 \rightarrow$ Lẻ).

Minh họa thanh ghi 8-bit

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
MSB							LSB
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
(128)	(64)	(32)	(16)	(8)	(4)	(2)	(1)

Endianness (Cách sắp xếp byte)

- ▶ **Little Endian:** Lưu Byte có trọng số thấp (LSByte) ở địa chỉ nhỏ nhất (ARM Cortex-M mặc định dùng).
- ▶ **Big Endian:** Lưu Byte có trọng số cao (MSByte) ở địa chỉ nhỏ nhất.

Tại sao stdint.h là bắt buộc?

- ▶ Các kiểu dữ liệu cơ bản như int, long thay đổi kích thước tùy vào kiến trúc CPU (8-bit, 16-bit, 32-bit).
- ▶ Dẫn đến tràn số (overflow) hoặc lãng phí RAM khi di chuyển code giữa các chip.
- ▶ stdint.h quy định rõ số bit của biến.

Kiểu dữ liệu	CPU 8-bit	CPU 32-bit
char	1 byte	1 byte
int	2 bytes	4 bytes
long	4 bytes	4 bytes

Table: So sánh kích thước kiểu dữ liệu

Khai báo biến chuẩn hóa (C99)

```
#include <stdint.h>

uint8_t  var_u8;   // 1 byte  (0..255)
int8_t   var_s8;   // 1 byte  (-128..127)
uint16_t var_u16;  // 2 bytes (0..65,535)
int16_t  var_s16;  // 2 bytes
uint32_t var_u32;  // 4 bytes (0..4.29B)
int32_t  var_s32;  // 4 bytes
```

Nguy hiểm: Sử dụng kiểu int trên vi điều khiển 8-bit (như AVR) để lưu giá trị lớn hơn 32,767 sẽ gây ra lỗi tràn số âm!

Macro (#define)

- ▶ Thay thế văn bản thô trước khi dịch.
- ▶ Không có kiểm tra kiểu, khó gỡ lỗi (debugger không thấy tên hằng số).
- ▶ Dùng cho cấu hình vật lý (địa chỉ, mask bit).

```
// Định nghĩa thông số vật lý
#define LED_PIN_0    0
#define GPIOA_BASE  0x40020000
```

Typedef Enum

- ▶ Định nghĩa các hằng số nguyên có kiểu.
- ▶ Trình biên dịch kiểm tra kiểu nghiêm ngặt và hiển thị tên biến khi debug.
- ▶ Dùng cho máy trạng thái (FSM) và trạng thái lỗi.

```
// Định nghĩa trạng thái hệ thống
typedef enum {
    STATE_OFF,
    STATE_RED,
    STATE_GREEN,
    STATE_BLUE
} LedState_t;
```

Struct — Gom nhóm tham số

- ▶ Gom nhóm các biến có liên quan logic thành một kiểu dữ liệu mới.
- ▶ Giúp cấu trúc chương trình sáng sủa, giảm số lượng đối số truyền vào hàm.
- ▶ Truyền hiệu quả hơn thông qua con trỏ (*pass-by-reference*).

Ví dụ ánh xạ phần cứng (fsm_demo.c)

```
typedef struct {  
    uint8_t pin;  
    const char* name;  
} Led_t;  
  
// Định nghĩa mảng LED hệ thống  
static const Led_t system_leds[] = {  
    {LED_PIN_0, "LED_RED"},  
    {1, "LED_GREEN"},  
    {2, "LED_BLUE"}  
};
```

Tại sao cần Bitwise?

- ▶ Vì điều khiển chỉ hiệu thanh ghi (thực chất là ô nhớ).
- ▶ Thay đổi trạng thái chân GPIO bằng cách tác động lên các bit tương ứng mà không ảnh hưởng tới các chân khác.

Bộ Macro thao tác Bit cơ bản

```
#define BIT_SET(REG, BIT) \
    ((REG) |= (1U << (BIT)))
#define BIT_CLEAR(REG, BIT) \
    ((REG) &= ~(1U << (BIT)))
#define BIT_TOGGLE(REG, BIT) \
    ((REG) ^= (1U << (BIT)))
#define BIT_CHECK(REG, BIT) \
    (((REG) & (1U << (BIT))) != 0)
```

Ví dụ thực hành (bit_demo.c)

```
volatile uint8_t GPIO_ODR = 0x00;
#define PIN_5 5

void set_pin_5(void) {
    GPIO_ODR |= (1 << PIN_5); // Set
}

void clear_pin_5(void) {
    GPIO_ODR &= ~(1 << PIN_5); // Clear
}

void toggle_pin_5(void) {
    GPIO_ODR ^= (1 << PIN_5); // Toggle
}

uint8_t check_pin_5(void) {
    return (GPIO_ODR & (1 << PIN_5)) ? 1 : 0;
}
```


Bản chất con trỏ

- ▶ Con trỏ thực chất là một biến lưu trữ **địa chỉ bộ nhớ** của biến khác.
- ▶ Cho phép truy cập và thay đổi trực tiếp vùng dữ liệu gốc.

Pass-by-value vs Pass-by-reference

- ▶ **Truyền tham trị:** Hàm nhận bản sao từ ngăn xếp (stack). Thay đổi bên trong hàm không ảnh hưởng tới biến gốc.
- ▶ **Truyền tham chiếu (dùng con trỏ):** Hàm nhận địa chỉ của đối số. Thay đổi qua con trỏ làm thay đổi trực tiếp biến gốc.

Code so sánh (fsm_demo.c)

```
// Truyền tham trị (Bảo vệ biến gốc)
void increment_by_value(int val) {
    val++; // Tăng bản sao trên stack
}

// Truyền con trỏ (Thay đổi biến gốc)
void increment_by_reference(int *ptr) {
    if (ptr != NULL) {
        (*ptr)++; // Công trực tiếp vào ô nhớ
    }
}

// Sử dụng trong main()
int var = 10;
increment_by_value(var); // var vẫn = 10
increment_by_reference(&var); // var bây giờ = 11
```



Nguyên lý cộng trừ con trỏ

- ▶ Phép toán cộng/trừ con trỏ phụ thuộc vào **kiểu dữ liệu** mà nó trỏ tới.
- ▶ Công thức dịch chuyển địa chỉ vật lý:
 $\text{Địa chỉ mới} = \text{Địa chỉ cũ} + (n \times \text{sizeof}(\text{Kiểu dữ liệu}))$
- ▶ Ví dụ: Cộng 1 vào con trỏ `uint32_t*` thực chất sẽ cộng 4 bytes vào địa chỉ vật lý của ô nhớ.

Ví dụ thực hành (3_pointer_arithmetic.c)

```
uint32_t arr[3] = {10, 20, 30};
uint32_t *ptr = arr;
// ptr trỏ vào arr[0] (ví dụ: 0x20000000)

ptr++;
// ptr trỏ vào arr[1] (địa chỉ tăng lên 4 bytes: 0x20000004)

uint8_t *byte_ptr = (uint8_t*)arr;
byte_ptr++;
// byte_ptr tăng lên đúng 1 byte (0x20000001)
```

Ứng dụng: Duyệt qua các vùng đệm bộ nhớ DMA hoặc mảng thanh ghi liên tiếp của thiết bị ngoại vi với tốc độ tối ưu.



Quy tắc đọc từ Phải qua Trái

- ▶ `const Type *ptr` (hoặc `Type const *ptr`): *Pointer to const Type*. Vùng dữ liệu bị khóa (→ không thể ghi đè dữ liệu `*ptr = x` ✗), địa chỉ của con trỏ có thể thay đổi để trỏ đi nơi khác ✓.
- ▶ `Type * const ptr`: *Const pointer to Type*. Địa chỉ con trỏ bị khóa (→ không thể trỏ đi nơi khác ✗), nhưng dữ liệu có thể chỉnh sửa tự do `*ptr = x` ✓.
- ▶ `const Type * const ptr`: *Const pointer to const Type*. Khóa cả dữ liệu lẫn địa chỉ (An toàn tối đa).

Ví dụ bảo vệ Driver API

```
// Dùng const pointer để bảo đảm hàm truyền dữ liệu  
// chỉ được phép ĐỌC, không được PHÉP SỬA dữ liệu truyền  
void UART_Transmit(const uint8_t *buffer, uint16_t size) {  
    for (uint16_t i = 0; i < size; i++) {  
        // buffer[i] = 0; // Lỗi Compile! Bảo vệ dữ liệu  
        USART1->DR = buffer[i];  
    }  
}
```

static Cục bộ (Local static)

- ▶ Khai báo bên trong hàm.
- ▶ Biến chỉ khởi tạo một lần duy nhất, giữ nguyên giá trị qua các lần gọi hàm tiếp theo.
- ▶ Phạm vi truy cập chỉ giới hạn trong hàm đó (Đóng gói dữ liệu).
- ▶ Ứng dụng: Đếm sự kiện, lưu trạng thái tạm thời, tạo System Tick.

static Toàn cục (Global static)

- ▶ Khai báo bên ngoài hàm ở file .c.
- ▶ Giới hạn tầm vực của biến/hàm chỉ trong file đó, ẩn đi đối với các file khác (tránh xung đột tên).

Ví dụ: System Tick (fsm_demo.c)

```
uint32_t generate_system_tick(void) {  
    // Lưu giá trị giữa các lần gọi hàm  
    // Biến này nằm ở phân data/bss chứ  
    // không  
    // nằm trên Stack!  
    static uint32_t tick_count = 0;  
    tick_count++;  
    return tick_count;  
}
```

Lưu ý: Biến static cục bộ không nằm trên vùng nhớ Stack mà được lưu tại phân vùng .data hoặc .bss của RAM.



Compiler thông minh như thế nào?

- ▶ Khi bật tối ưu (-O2, -O3), trình biên dịch sẽ tối giản mã lệnh.
- ▶ Nó sẽ lưu giá trị của biến vào thanh ghi CPU tốc độ cao thay vì đọc từ RAM liên tục, hoặc xóa các biến/vòng lặp không sinh ra kết quả đầu ra (Dead Code).

volatile — Lệnh cấm tối ưu hóa

- ▶ Báo cho trình biên dịch biết biến này có thể thay đổi bất ngờ bởi phần cứng bên ngoài hoặc luồng khác.
- ▶ Bắt buộc CPU phải đọc/ghi trực tiếp giá trị vào ô nhớ RAM trong mỗi lần thực thi lệnh.

3 trường hợp bắt buộc sử dụng volatile

1. Thanh ghi phần cứng (ví dụ ADC, GPIO).
2. Biến chia sẻ giữa hàm chính và ISR.
3. Vòng lặp delay phần mềm.

Không dùng volatile (Bị tối ưu hóa xóa bỏ)

```
void loop_without_volatile(void) {  
    uint32_t count = 0;  
    while (count < 1000) {  
        count++;  
    }  
}
```

Assembly dịch ngược (-O2):

```
loop_without_volatile:  
    ret    ; Bi compiler xoa so hoan toan!
```

Có dùng volatile (Giữ nguyên vòng lặp delay)

```
void loop_with_volatile(void) {  
    volatile uint32_t count = 0;  
    while (count < 1000) {  
        count++;  
    }  
}
```

Assembly dịch ngược (-O2):

```
loop_with_volatile:  
    mov     DWORD PTR [rsp-4], 0    ; Ghi xuong RAM  
.L3:  
    mov     eax, DWORD PTR [rsp-4]; Doc tu RAM  
    cmp     eax, 999  
    jg      .L4  
    add     eax, 1  
    mov     DWORD PTR [rsp-4], eax; Ghi lai RAM  
    jmp     .L3  
.L4:  
    ret
```



Không dùng volatile (Chỉ đọc RAM 1 lần)

```
uint8_t read_without_volatile(uint8_t *reg)
{
    uint8_t val1 = *reg;
    uint8_t val2 = *reg; // Copy tu thanh ghi CPU
    return val1 + val2;
}
```

Assembly dịch ngược (-O2):

```
read_without_volatile:
    movzx    eax, BYTE PTR [rdi] ; Doc tu dia chi 1 lan
    add      eax, eax             ; Nhan doi lun
    ret
```

Có dùng volatile (Đọc RAM 2 lần bắt buộc)

```
uint8_t read_with_volatile(volatile uint8_t
    *reg) {
    uint8_t val1 = *reg;
    uint8_t val2 = *reg; // Bat buoc doc tu RAM
    return val1 + val2;
}
```

Assembly dịch ngược (-O2):

```
read_with_volatile:
    movzx    eax, BYTE PTR [rdi] ; Doc tu dia chi lan 1
    movzx    edx, BYTE PTR [rdi] ; Doc tu dia chi lan 2
    add      eax, edx
    ret
```

Sai lầm: Sử dụng delay_ms()

- ▶ Hàm delay() chặn hoàn toàn CPU (blocking). CPU chạy vòng lặp vô nghĩa và bỏ lỡ các sự kiện quan trọng (nhấn nút, ngắt ngoại vi).

Tư duy Non-blocking với FSM

- ▶ Chia hệ thống thành các trạng thái rời rạc (**States**).
- ▶ Thăm dò sự kiện (**Events**) liên tục.
- ▶ Chuyển đổi trạng thái (**Transition**) nhanh chóng và trả CPU cho tác vụ khác.

Cấu trúc FSM tiêu chuẩn trong C

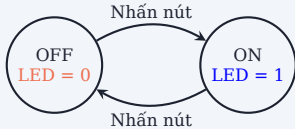
```
typedef enum {  
    STATE_IDLE,  
    STATE_PROCESSING,  
    STATE_ERROR  
} SystemState_t;  
  
void process_fsm(void) {  
    static SystemState_t state = STATE_IDLE;  
    switch (state) {  
        case STATE_IDLE:  
            if (event_trigger) state = STATE_PROCESSING;  
            break;  
        case STATE_PROCESSING:  
            // Thực hiện công việc nhanh  
            state = STATE_IDLE;  
            break;  
        case STATE_ERROR:  
            break;  
    }  
}
```


Tiêu chí	Moore FSM	Mealy FSM
Ngõ ra (Output)	Chỉ phụ thuộc vào Trạng thái hiện tại	Phụ thuộc vào Trạng thái + Đầu vào
Vị trí Ngõ ra	Nằm trong ô trạng thái (State Bubble)	Nằm trên mũi tên chuyển dịch
Thời gian Phản hồi	Chậm hơn (phải đợi chuyển sang trạng thái mới)	Nhanh hơn (đáp ứng tức thời khi có đầu vào)
Số lượng Trạng thái	Thường cần nhiều trạng thái hơn	Thường cần ít trạng thái hơn
Hiện tượng Nhiễu (Glitch)	An toàn , không bị ảnh hưởng bởi nhiễu đầu vào	Nguy cơ bị nhiễu ở ngõ ra do đầu vào run

Table: So sánh giữa Moore và Mealy FSM

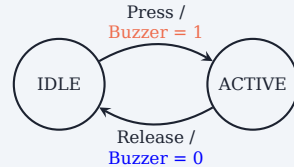


Moore FSM: Đèn hành lang (LED)



- Ngõ ra (LED) gắn liền với **Trạng thái**.

Mealy FSM: Còi cảnh báo (Buzzer)



- Ngõ ra (Buzzer) đi kèm **đường chuyển dịch**.



Tại sao cần Debouncing?

- ▶ Nút nhấn cơ học bị rung động cơ học (dội phím) trong khoảng 10 ~ 50ms khi nhấn hoặc nhả.
- ▶ Nếu đọc trực tiếp, CPU sẽ nhận lầm là người dùng nhấn liên tiếp nhiều lần.

Mô hình FSM 4 trạng thái

1. **BUTTON_STATE_RELEASED:** Chờ nút nhấn.
2. **BUTTON_STATE_DEBOUNCE_PRESS:** Phát hiện nút nhấn, chờ 50ms để xác minh.
3. **BUTTON_STATE_PRESSED:** Xác nhận nhấn thật.
4. **BUTTON_STATE_DEBOUNCE_RELEASE:** Phát hiện nút nhả, chờ 50ms để xác minh.

```
bool process_button_fsm(uint32_t current_tick) {
    static ButtonState_t state = BUTTON_STATE_RELEASED;
    static uint32_t state_entry_tick = 0;
    bool button_event_pressed = false;
    const uint32_t DEBOUNCE_TICKS = 5; // -50ms (neu 1 tick = 10ms)

    switch (state) {
        case BUTTON_STATE_RELEASED:
            if (raw_button_input == true) {
                state = BUTTON_STATE_DEBOUNCE_PRESS;
                state_entry_tick = current_tick;
            } break;
        case BUTTON_STATE_DEBOUNCE_PRESS:
            if (raw_button_input == false) state = BUTTON_STATE_RELEASED;
            else if (current_tick - state_entry_tick >= DEBOUNCE_TICKS) {
                state = BUTTON_STATE_PRESSED;
                button_event_pressed = true; // Bao su kien click confirmed
            } break;
        case BUTTON_STATE_PRESSED:
            if (raw_button_input == false) {
                state = BUTTON_STATE_DEBOUNCE_RELEASE;
                state_entry_tick = current_tick;
            } break;
        case BUTTON_STATE_DEBOUNCE_RELEASE:
            if (raw_button_input == true) state = BUTTON_STATE_PRESSED;
            else if (current_tick - state_entry_tick >= DEBOUNCE_TICKS) {
                state = BUTTON_STATE_RELEASED;
            } break;
    }
    return button_event_pressed;
}
```



Yêu cầu bài toán

- ▶ Chuyển đổi trạng thái LED (RED → GREEN → BLUE → OFF) tuần tự.
- ▶ Tự động chuyển đổi sau mỗi 20 ticks (*timeout*).
- ▶ Hoặc chuyển đổi ngay lập tức khi phát hiện sự kiện nút nhấn từ hàm debounce.
- ▶ **Tuyệt đối không sử dụng hàm delay!**

Nguyên lý hoạt động

- ▶ Sử dụng thanh ghi GPIOA_ODR để ghi trạng thái chân LED.
- ▶ Dùng hàm BIT_SET và BIT_CLEAR để điều khiển bật/tắt chính xác các chân.
- ▶ Giữ mốc thời gian chuyển đổi qua biến `static last_toggle_tick`.



```
void process_led_fsm(uint32_t current_tick, bool trigger_next) {
    static LedState_t state = LED_STATE_OFF;
    static uint32_t last_toggle_tick = 0;
    const uint32_t AUTO_SWITCH_TICKS = 20;

    bool time_to_switch = (current_tick - last_toggle_tick >= AUTO_SWITCH_TICKS);
    if (trigger_next || time_to_switch) {
        last_toggle_tick = current_tick;

        // Tat het tat ca cac LED truoc
        BIT_CLEAR(GPIOA_ODR, LED_PIN_0);
        BIT_CLEAR(GPIOA_ODR, LED_PIN_1);
        BIT_CLEAR(GPIOA_ODR, LED_PIN_2);

        switch (state) {
            case LED_STATE_OFF:
                state = LED_STATE_RED;
                BIT_SET(GPIOA_ODR, LED_PIN_0); break;
            case LED_STATE_RED:
                state = LED_STATE_GREEN;
                BIT_SET(GPIOA_ODR, LED_PIN_1); break;
            case LED_STATE_GREEN:
                state = LED_STATE_BLUE;
                BIT_SET(GPIOA_ODR, LED_PIN_2); break;
            case LED_STATE_BLUE:
                state = LED_STATE_OFF; break;
        }
    }
}
```

- ▶ **Kiến trúc máy tính & Bit:** Hiểu được Von Neumann bottleneck, lợi thế Harvard và cách biểu diễn byte trong bộ nhớ dựa trên LSB, MSB, Endianness.
- ▶ **stdint.h & Con trỏ nâng cao:** Chuẩn hóa kích thước kiểu dữ liệu. Sử dụng phép toán con trỏ và từ khóa `const` để viết mã nguồn API Driver an toàn, hiệu quả.
- ▶ **volatile:** Ngăn chặn tối ưu hóa sai của compiler đối với các thanh ghi phần cứng hoặc luồng cờ ngắt.
- ▶ **FSM & Non-blocking:** Lập trình máy trạng thái kết hợp System Tick là giải pháp duy nhất giải phóng CPU khỏi hàm `delay()` lãng phí.

Tài nguyên thực hành thêm:

- ▶ Tra cứu sơ đồ thanh ghi GPIO trong file Datasheet của STM32F4 hoặc ESP32.
- ▶ Biên dịch và chạy thử trình mô phỏng FSM trực tiếp qua file `fsm_demo.c`.

Thank You

Acknowledgements

Cảm ơn các bạn đã lắng nghe. Hãy thực hành viết mã nguồn sạch, tối ưu và chuẩn hóa cho hệ thống nhúng!